

RAPPORT DE STAGE

DUPÉ Timéo

Du 18 Mai au 19 Juin 2026

Tuteur de stage : Monsieur Frédéric LANOË

Enseignant référent : Madame CARBONNIER

Établissement / Formation : La Joliverie – BTS SIO

Classe : 1SIO B

Entreprise d'accueil : IWAN – 26 Rue des Déportés Résistants 44110 Châteaubriant

Sommaire

<i>I – Introduction</i>	<i>1</i>
<i>II-Présentation de l'entreprise</i>	<i>1</i>
A. IWAN c'est quoi ?	1
B. Mon intégration dans l'entreprise	2
<i>III-Développement de l'application</i>	<i>2</i>
A. Préparation de l'environnement.....	2
B. Développement de l'application de ticketing.....	3
<i>IV – Bilan technique.....</i>	<i>9</i>
<i>V – Conclusion</i>	<i>10</i>

I – Introduction

Face à l'augmentation drastique de parcs informatiques des entreprises, la demande d'assistance et de solutions sur mesure ne cesse d'augmenter. Ainsi, de nombreuses sociétés de développement de logiciels applicatifs voient le jour. C'est le cas d'IWAN, une entreprise spécialisée dans les logiciels de gestion et la création d'applications sur mesure.

Dans le cadre de ma formation en BTS SIO, il m'a été demandé de réaliser un stage dans une entreprise informatique, afin d'acquérir une expérience professionnelle et de nous faire une idée de la vie en entreprise. Ainsi, IWAN m'a offert l'opportunité de me faire découvrir le monde de l'entreprise en me prenant en stage du 18 mai au 19 juin 2026.

Au cours de ce stage ma mission était de réaliser une application de ticketing afin de faciliter la gestion des incidents et des demandes des clients d'IWAN. Cette application ayant pour but d'être utilisée, il fallait donc qu'elle soit entièrement opérationnelle à la fin de mon stage. Il a donc fallu passer par une phase de réflexion avant de commencer à la réaliser.

II-Présentation de l'entreprise

A. IWAN c'est quoi ?

Iwan est une SARL (Société à responsabilité limitée) fondée par Marlène ESNAULT et Frédéric LANOE en 2019. Basée à Châteaubriant, elle vise des clients dans tous les pays francophones mais la plupart de ses clients se situent en France métropolitaine.

Elle est spécialisée dans le développement de logiciels applicatifs et s'adapte aux demandes des clients qui peuvent avoir des besoins spécifiques. Ils ont notamment réalisé un logiciel pour gérer et mettre en relation les demandeurs et offreurs d'emploi (GOA). Un logiciel de devis, commande et facture (ERP), ainsi qu'un logiciel de réservation pour les vacances adapté (Resavac). Une partie de leurs clients est aussi présente pour leur logiciel de caisse et de gestion de stock.

IWAN est composé de 2 associés : Marlène ESNAULT qui occupe le poste de Technico-Commerciale et Formatrice, et de Frédéric LANOE qui lui est Développeur. A la création, Frédéric avait les logiciels et Marlène les contacts clients, ils ont donc décidé de fonder IWAN ensemble.

B. Mon intégration dans l'entreprise

Lors de mon arrivée dans l'entreprise, j'ai directement été accueilli par Marlène et Frédéric ainsi que par un autre stagiaire, Antoine. Étant une petite entreprise et ayant déjà un stagiaire, ils n'avaient pas d'ordinateur à me fournir, j'ai donc passé mon stage sur mon propre PC, ce qui m'a permis de garder mes habitudes. Afin de réaliser ce projet seul, j'ai utilisé Visual Studio Code, XAMPP pour le serveur local et Looping pour la modélisation. Une fois installé, Frédéric m'a expliqué ce que j'allais faire lors de mon stage, pourquoi ils avaient besoin d'une nouvelle application de ticketing et les besoins auxquels cette nouvelle application devait répondre.

Pendant toute la durée de mon stage, la conversation avec les autres employés était aisée, étant tous dans la même pièce, je pouvais facilement poser des questions. Tous les jours, je montrais mes avancées et je posais quelques questions pour orienter mon travail. Le stage se faisait en présentiel de 9h à 12h puis de 14h à 18h.

III-Développement de l'application

A. Préparation de l'environnement

Avant mon arrivée, Antoine, qui était déjà présent depuis quelques semaines, avait réalisé une maquette sur Figma des différentes pages de l'application afin que je ne parte pas de zéro. J'ai donc commencé par me familiariser avec Figma. Afin d'organiser mon projet comme il le faut et faciliter la reprise par une autre personne après, j'ai dû apprendre et comprendre la logique du MVC (Modèle Vue Controller). Si au début j'ai eu du mal à m'y habituer, j'ai très vite compris l'importance et la praticité de cette architecture. Le MVC permet de séparer mon code en différentes parties : le Modèle qui gère toutes les interactions avec la base de données, la Vue qui est ce que l'utilisateur voit (html) et le Contrôleur qui contient la logique et qui met à jour le modèle ou la vue en fonction des actions de l'utilisateur. Afin de bien comprendre quelle partie de mon code mettre, Antoine m'a réalisé un schéma afin de m'expliquer les différentes parties de du projet.

Le routeur (index.php) permet de rediriger au bon endroit avec une URL de la forme `index.php ?page=.....`

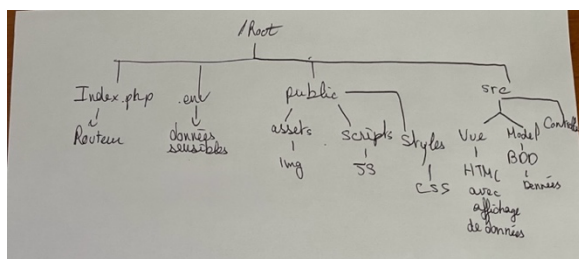


Figure 1 : Arborescence du projet illustrant la mise en place de l'architecture MVC

Le .env permet de stocker les données sensibles comme un mot de passe ou l'url de la base de données.

Dans le dossier public on va mettre tout ce qui va permettre de changer le style de la page, le java script, le css ou les assets.

Enfin dans src c'est là que l'on applique le

MVC. Lors de l'appropriation de ce model j'ai posé pas mal de question ce qui ma permit de bien cerner le fonctionnement.

B. Développement de l'application de ticketing

A quoi va servir cette application ?

Ma mission est de développer une application afin de relier les clients à IWAN. Si lors de l'utilisation des logiciels d'IWAN les clients rencontrent des bugs, ont des problèmes ou veulent une amélioration ou nouvelle fonctionnalité, ils peuvent donc créer un ticket. Comme cela, chaque demande est numérotée et centralisée au même endroit. Cette application va aussi permettre de classer les demandes par niveau d'urgence afin de résoudre les problèmes bloquants en priorité. IWAN ayant aussi besoin de réaliser des suivis lors des appels, une section pour créer des suivis pour les clients sera aussi nécessaire afin d'assurer une centralisation de toutes les informations, ainsi qu'un suivi et une traçabilité irréprochable.

Réalisation techniques et fonctionnalités clés

Conception de MLD

Pour réaliser une application de ce type, il est indispensable d'avoir une base de données. Cette base n'existant pas, il a fallu partir de 0 et j'avais comme consigne d'évaluer les besoins et de créer la base en fonction. De l'évaluation des besoins au dictionnaire de données, puis à la réalisation du MLD sur looping, j'en suis arrivé à une architecture me permettant de bien commencer mon projet, même si des modifications seront apportées par la suite afin de répondre parfaitement aux besoins.

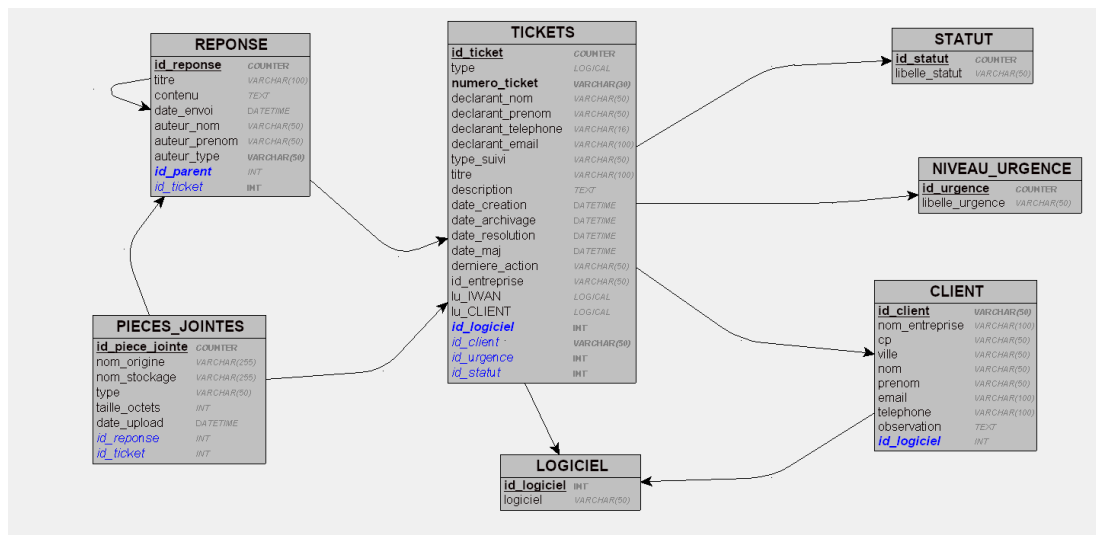


Figure 2: Logiciel Looping : Dernière version du MLD du site de ticketing

L'accès au site de ticketing se fera par l'espace client des entreprises, elles seront déjà connectées et auront un identifiant unique. Cet identifiant sera mis dans l'URL et c'est de cette façon que l'on identifiera le client.

Interfaces dynamiques et requêtes asynchrones (AJAX)

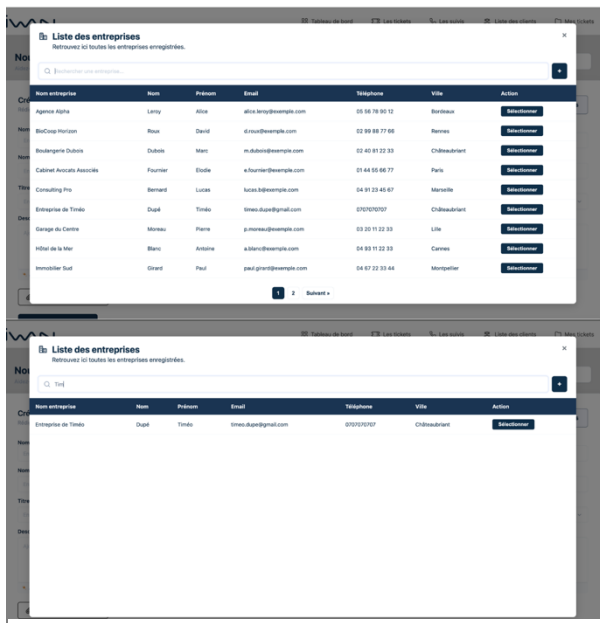


Figure 3 : Interface de recherche dynamique préservant les données déjà saisies.

millisecondes et si l'utilisateur ne tape rien d'autre, alors une requête interroge le serveur en arrière-plan qui renvoie les données au format JSON, et met à jour le résultat en temps réel sans avoir à recharger la page, ce qui permet de garder les informations ou notes déjà rentrées sur le ticket avant la sélection de l'entreprise.

Afin d'éviter de recharger la page à chaque requête, ce qui ralentit la navigation et de répondre pleinement aux besoins d'IWAN, j'ai beaucoup utilisé du JavaScript avec l'API Fetch pour réaliser des requêtes asynchrones. Par exemple, lors de la création de tickets par IWAN, j'ai développé une modale pour rechercher des entreprises dans la base de données afin de préremplir le formulaire. Sans utiliser AJAX, il faudrait recharger la page à chaque action ou recherche dans le filtre des entreprises. Or, avec le JavaScript que j'ai ajouté à chaque fois que l'utilisateur tape une lettre, on attend 300

index.php?page=nouveau_ticket&action=reche...	200	fetch	Nouveau_ticket.js:551	2.4 kB	39 ms
index.php?page=nouveau_ticket&action=reche...	200	fetch	Nouveau_ticket.js:551	0.7 kB	34 ms

Figure 4: Analyse de la requête asynchrone (Fetch) et de la réponse JSON du serveur.

Gestion des pièces jointes

La gestion des pièces jointes liées aux tickets et aux réponses a fait l'objet d'un développement spécifique. Lors de la création de ticket, il est fréquent de se retrouver avec des captures d'écran qui peuvent toutes avoir le même nom, les stocker telles quelles dans le serveur rendrait impossible leur récupération. C'est pour cela qu'à leur upload sur le serveur, je génère un nom unique avec la fonction PHP `uniqid()` (exemple : 6a27c6be4d498.pdf) tout en gardant l'extension. Pour nettoyer le nom de tous ses caractères spéciaux, j'utilise un `preg_replace`. Une fois fait, les informations sont stockées dans la base de données ainsi que le nom sur le serveur afin de permettre son téléchargement lors de la consultation du ticket. J'ai aussi développé un script qui analyse l'extension du fichier afin de générer un svg attribué à chaque fichier. J'ai aussi optimisé, à la demande de mon tuteur, l'affichage des pièces jointes. Si le fichier (PDF, png, JPG) est lisible par le navigateur, alors je lui attribue `target= "_blank"` afin de l'ouvrir dans un nouvel onglet sans le télécharger. Si en revanche il s'agit d'un fichier que le navigateur ne peut pas ouvrir, alors je lui mets un attribut `download`, ce qui force le téléchargement avec son nom d'origine restauré.

```
<?php if ($est_lisible): ?>
target="_blank"
title="Ouvrir la pièce jointe"
<?php else: ?>
download="<?= htmlspecialchars($pj['nom_origine']) ?>"
title="Télécharger"
<?php endif; ?>
```

Figure 6: Gestion du `_blank` ou `download`

Figure 5: Affichage des pièces jointes avec les logos SVG généré automatiquement en fonction de l'extension du fichier

		id_piece_jointe	nom_origine	nom_stockage	type	taille_octets	date_upload	id_reponse	id_ticket
☐	Éditer Copier Supprimer	45	photo_test.JPG	6a3830be4c6aa.JPG	image/jpeg	4863512	2026-06-21 20:43:10	NULL	122
☐	Éditer Copier Supprimer	46	Liste_des_entreprises.pdf	6a3830be4cab1.pdf	application/pdf	1146593	2026-06-21 20:43:10	NULL	122
☐	Éditer Copier Supprimer	47	Note_de_veille_Informatique.docx	6a3830be4cde6.docx	application/vnd.openxmlformats-officedocument.word...	18724	2026-06-21 20:43:10	NULL	122

Figure 7: Informations du fichiers entrées dans la base de données

Génération de documents PDF

À l'origine, tous les tickets restaient uniquement sur le site, il n'y avait aucun moyen de sortir une version propre. Afin de répondre à un besoin de traçabilité, j'ai donc intégré une fonctionnalité pour exporter les tickets au format PDF. Pour réaliser cet export, j'ai utilisé l'outil Composer afin d'installer la librairie PHP Dompdf. J'ai conçu une vue HTML/CSS spécifique à la création du PDF au format A4 prêt à être imprimé. Le contrôleur récupère toutes les informations du ticket et des réponses dans la base de données et

les affiche dans la vue. Une fois fait Dompdf, converti la vue HTML en un fichier PDF prêt à être téléchargé.



Figure 9 : Vu PDF du ticket

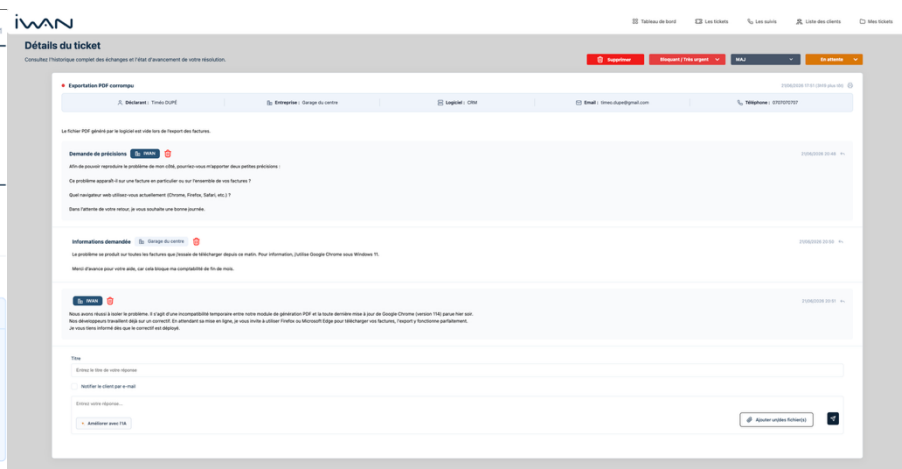


Figure 8 : Vu du ticket sur le site web

Intégration de modèle de langage (API OpenAI)

Afin de faciliter la rédaction des tickets pour les clients comme pour IWAN, il m'a été demandé d'ajouter une fonctionnalité pour reformuler et corriger les fautes d'orthographe et de grammaire avec de l'IA (chat GPT). Lorsque l'utilisateur clique sur le bouton " Améliorer avec l'IA", le texte est envoyé via Fetch à mon contrôleur PHP qui exécute les fonctions de mon modèle afin d'envoyer la requête HTTPS vers le serveur d'OpenAI en utilisant la clé d'API stockée dans le .env. Seul le message est envoyé au modèle de langage (LLM), aucune info permettant d'identifier le client n'est transmise. Une fois que le serveur renvoie le texte corrigé, une fenêtre modale s'ouvre pour montrer le texte modifié par le LLM et que l'on puisse choisir de remplacer notre texte original par l'IA ou non, l'utilisateur garde alors un contrôle total sur le texte généré.

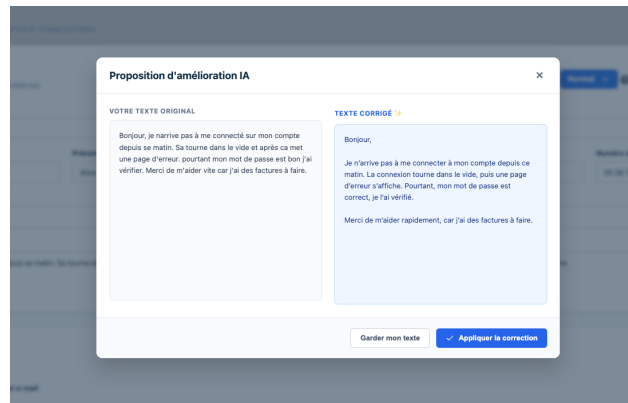


Figure 10 : Fenêtre modale de prévisualisation offrant à l'utilisateur un contrôle total avant l'application des corrections générées par l'IA.

Envoie de mail automatique

L'une des fonctionnalités les plus importantes est l'envoi de mail automatique, que ce soit pour prévenir IWAN d'un nouveau ticket ou pour prévenir le client d'une réponse, d'un nouveau ticket ou suivi créé à son nom. Pour envoyer des mails en PHP, il y a deux méthodes : la fonction mail() de base, mais elle finit souvent dans les spams, et la librairie PHPMailer que j'ai donc utilisée. Afin d'envoyer ces mails, je me connecte avec le protocole SMTP au serveur de messagerie de IONOS avec un mail et un mot de passe stockés dans le .env, le mail client lui est récupéré dans le formulaire du ticket ou du suivi. Pour la sécurité, j'ai utilisé ENCRYPTION_SMTPS afin que les données du mail voyagent de manière chiffrée du début à la fin. Afin de gérer l'erreur lors de l'envoi, toutes mes actions sont dans un try qui réalise la connexion au serveur et l'envoi du mail. Si à un moment quelque chose bloque, que ce soit un mot de passe erroné ou que le serveur est indisponible, ma fonction récupère l'erreur avec le catch afin de maintenir la stabilité de l'application. Comme pour la génération du PDF, j'ai dû créer des templates pour les mails, dans lesquels j'injecte les données afin de fournir aux clients et à IWAN des mails professionnels et structurés.

```
function envoyer_mail(string $destinataire, string $sujet, string $corps_html): bool
{
    $destinataire = filter_var(trim($destinataire), FILTER_VALIDATE_EMAIL);
    if (!$destinataire) return false;

    $mail = new PHPMailer(true);

    try {
        // CONNEXION SMTP IONOS
        $mail->isSMTP();
        $mail->Host = SMTP_HOST;
        $mail->SMTPAuth = true;
        $mail->Username = MAIL_FROM;
        $mail->Password = SMTP_PASSWORD;

        // ENCRYPTION_SMTPS pour le SSL/TLS pur
        $mail->SMTPSecure = PHPMailer::ENCRYPTION_SMTPS;
        $mail->Port = SMTP_PORT;

        $mail->CharSet = 'UTF-8';

        // EXPÉDITEUR ET DESTINATAIRE
        $mail->setFrom(MAIL_FROM, MAIL_FROM_NAME);
        $mail->addAddress($destinataire);

        // CONTENU
        $mail->isHTML(true);
        $mail->Subject = $sujet;
        $mail->Body = $corps_html;

        $mail->send();
        return true;
    } catch (Exception $e) {
        error_log("Echec SMTP Iwan Assistance : " . $mail->ErrorInfo);
        return false;
    }
}
```

Figure 11 : Configuration sécurisée du protocole SMTP via la bibliothèque PHPMailer avec gestion de l'erreur (try/catch)

Iwan Assistance

Nouveau ticket créé

Entreprise	Garage du centre
Déclarant	Timéo Dupé
Email	timeo.dupe@gmail.com
Urgence	Urgent
Date	21/06/2026 à 21:38

Erreur de calcul taxe

Description

La TVA n'est pas correctement appliquée sur les produits importés dans l'ERP.

Voir la ticket

Figure 12 : Modèle d'e-mail transactionnel (template HTML) envoyé au client, intégrant un récapitulatif clair et un bouton d'accès direct à la plateforme.

Outils utilisés

Pour commencer, j'ai utilisé Visual Studio Code (VS Code), un éditeur de code simple et très utilisé. Ayant pris mes marques sur ce logiciel, je suis familière avec tous les raccourcis et des extensions comme "Prêteur" pour organiser mon code automatiquement.

Ensuite, j'ai utilisé Looping pour modéliser la base de données (Conception d'un MLD). Une fois que l'on a réalisé le dictionnaire de données, cet outil permet de créer des entités avec différents types de données : entiers, chaîne de caractères et autres, ainsi que des associations entre les entités (relations). Ce que l'on appelle un modèle logique de données (MLD).

Pour la partie interface, j'ai utilisé l'outil de design collaboratif Figma. Avant mon arrivée, Antoine (l'autre stagiaire) y avait déjà réalisé les maquettes interactives des différentes pages de l'application. Dans ce projet, je me suis donc familiarisé avec cet outil qui permet de visualiser l'ergonomie et le parcours utilisateur avant de commencer à coder. Figma m'a donc servi de guide visuel pendant toute la partie de création des vues en HTML CSS, le tout en respectant la charte graphique d'IWAN.

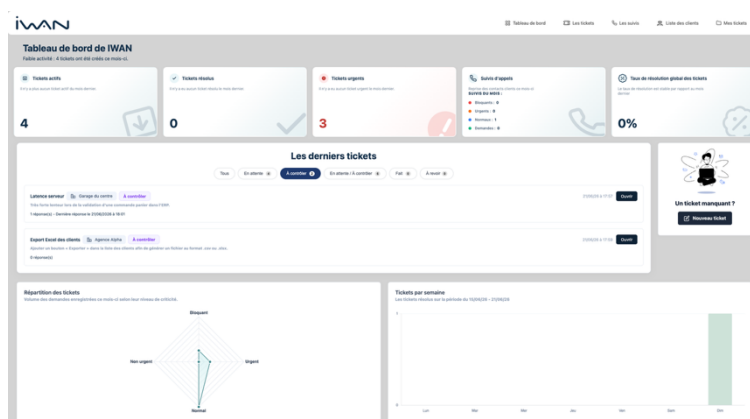


Figure 13 : Vu du tableau de bord d'IWAN de l'application WEB réalisé

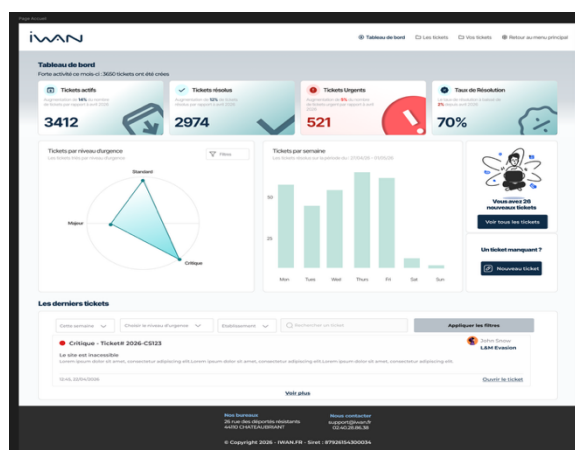


Figure 14 : Maquette du tableau de bord d'IWAN réalisé sur Figma

Afin de développer et de tester l'application web en toute autonomie, j'ai utilisé le logiciel XAMPP. Ce logiciel m'a fourni les éléments essentiels au développement de mon application : le serveur web Apache pour interpréter et exécuter mon code PHP, le système de gestion de base de données relationnelle (SGBDR) MySQL/MariaDB. De plus, l'interface phpMyAdmin m'a grandement facilité l'administration de ma base de données, me permettant d'exécuter des requêtes SQL de test pour vérifier l'intégrité de mes données.

Une fois le développement local terminé et l'application validée, il a fallu procéder à sa mise en production afin de la rendre accessible aux clients d'IWAN. L'entreprise m'a pour cela mis à disposition l'infrastructure d'hébergement web de IONOS à laquelle ils souscrivent afin d'y déployer mon application web. Pour transférer le code source de mon projet (script PHP, ressources statiques, vues) de mon ordinateur vers le serveur distant, j'ai utilisé FileZilla, un client FTP (File Transfert Protocol). Cet outil m'a permis de me connecter de manière sécurisée à l'espace d'hébergement et d'y synchroniser l'arborescence MVC ainsi que tous mes fichiers. En parallèle, j'ai dû exporter ma base de données locale pour l'importer sur le serveur SGBD distant de IONOS, finalisant ainsi le déploiement du projet et la mise à disposition de l'application aux clients d'IWAN.

IV – Bilan technique

Au terme de mon stage, l'application a été rendue dans une version entièrement fonctionnelle et prête pour la production. L'interface client et l'interface pour IWAN répondaient parfaitement aux exigences fixées dans le cahier des charges.

Cependant, avec davantage de temps, plusieurs points d'amélioration techniques ou ergonomiques pourraient être largement améliorés pour optimiser le traitement des tickets par l'équipe :

- **L'optimisation mobile (Responsive Design) :** Actuellement, lors de la consultation de l'application web sur smartphone, on remarque certaines limites, en particulier au niveau des tableaux pour lister les tickets et les clients. Ceux-ci s'affichent comme sur un écran d'ordinateur et dépassent donc de l'écran. Il a été vu avec mon tuteur que la meilleure façon pour pallier cela serait de réduire le nombre de colonnes sur le tableau sur mobile pour y laisser uniquement le titre et le bouton pour ouvrir la page de détails du ticket.

- **Envoie des mails asynchrone** : Actuellement, lors de l'envoi des mails, la page charge jusqu'à ce que la connexion au serveur SMTP et l'envoi du mail via PHPMailer soient terminés avant de recharger la page. Cela est fonctionnel, mais n'est pas agréable pour l'interface utilisateur et ralentit considérablement le traitement des tickets par IWAN, car l'envoi peut parfois prendre du temps. Pour pallier ce problème, il faudrait rendre l'envoi des mails asynchrone, par exemple en réalisant un script qui enverrait les mails en arrière-plan, ce qui ne dérangerait pas l'interface utilisateur.
- **Migration vers un framework PHP** : Le projet a été réalisé en PHP natif avec une architecture MVC. Sur le long terme, une évolution logique serait de migrer le code source de l'application web vers un framework comme Symfony ou Laravel. Cela faciliterait la maintenance et renforcerait la sécurité native.
- **Harmonisation et optimisation de l'interface de suivi** : A l'issue des tests utilisateurs avec l'équipe d'IWAN, il est apparu que l'interface actuelle des suivis, basée sur un texte, manquait de fluidité et n'était pas adaptée pour un traitement rapide et une bonne compréhension. J'ai augmenté la taille de la texture pour l'améliorer, mais il serait nécessaire de modifier ce fonctionnement. On pourrait par exemple utiliser une vue chronologique comme un fil de discussion afin de faciliter le suivi. Cette évolution permettrait de mieux structurer l'historique des interactions et de faciliter les réponses aux demandes des clients

V – Conclusion

Par conséquent, ce stage m'a énormément appris et guidé sur mon apprentissage. Le travail au sein d'une entreprise telle qu'IWAN m'a donné une bonne impression de ce qu'est le travail de développeur fullstack. Je suis devenu beaucoup plus autonome et j'ai appris à structurer un projet, mon travail et à organiser les différentes tâches. J'ai appris à effectuer des recherches efficaces et à mieux utiliser les outils indispensables à un développeur fullstack. J'ai aussi pu mettre en pratique des compétences vues en cours : l'évaluation des besoins afin de réaliser un dictionnaire de données, la réalisation d'un MLD, la configuration de mon environnement de développement avec VS Code. La création de requêtes SQL complexes ainsi que les sécurités importantes vues en cours comme l'injection SQL ou encore XSS. Lors de ce stage, je suis sorti de ma zone de confort et ai dû monter un projet en ayant comme information que le minimum, ce qui était particulièrement compliqué pour moi avant.